

Event#

<https://pytorch.org/docs/stable/generated/torch.mtia.Event.html>

Description:

Query and record Stream status to identify or control dependencies across Stream and measure timing. `device` (`torch.device`, optional) – the desired device for the Event. If not given, the current accelerator type will be used. `enable_timing` (`bool`, optional) – indicates if the event should measure time (default: `False`) `blocking` (`bool`, optional) – if `True`, `wait()` will be blocking (default: `False`) `interprocess` (`bool`) – if `True`, the event can be shared between processes (default: `False`)

Function Signature

```
class torch.mtia.Event(device=None, *, enable_timing=False, blocking=False, interprocess=False)#
```

Parameters

Returns

Return type

```
elapsed_time(end_event) → float#
```

Returns:

bool

Code Examples

```
>>> event = torch.Event()  
>>> e_cuda = torch.Event(device='cuda')
```

```
>>> s_cuda = torch.Stream(device='cuda')  
>>> e1_cuda = s_cuda.record_event()  
>>> e2_cuda = s_cuda.record_event()  
>>> ms = e1_cuda.elapsed_time(e2_cuda)
```

```
>>> s_cuda = torch.Stream(device='cuda')  
>>> e_cuda = s_cuda.record_event()  
>>> e_cuda.query()  
True
```

```
>>> e_cuda = torch.Event(device='cuda')  
>>> e_cuda.record()
```

```
>>> s_cuda = torch.Stream(device='cuda')
>>> e_cuda = s_cuda.record_event()
>>> e_cuda.synchronize()
```

```
>>> s1_cuda = torch.Stream(device='cuda')
>>> s2_cuda = torch.Stream(device='cuda')
>>> e_cuda = s1_cuda.record()
>>> e_cuda.wait(s2)
```

Notes & Warnings

WARNING

Warning Both blocking and interprocess are not supported right now and are noops.

Detailed Documentation

Documentation

Query and record Stream status to identify or control dependencies across Stream and measure timing.

device (torch.device, optional) – the desired device for the Event. If not given, the current accelerator type will be used.

`enable_timing` (bool, optional) – indicates if the event should measure time (default: False)

`blocking` (bool, optional) – if True, `wait()` will be blocking (default: False)

`interprocess` (bool) – if True, the event can be shared between processes (default: False)

Both `blocking` and `interprocess` are not supported right now and are noops.

An `torch.Event` object.

Returns the elapsed time in milliseconds between when this event and the `end_event` are each recorded via `torch.Stream.record_event()`.

`end_event` (`torch.Event`) – The ending event has been recorded.

Time between starting and ending event in milliseconds.

Check if the stream where this event was recorded already moved past the point where the event was recorded. Always returns True if the Event was not recorded.

A boolean indicating if all work currently captured by event has completed.

Record the event in a given stream. The stream's device must match the event's device. This function is equivalent to `stream.record_event(self)`.

`stream` (`torch.Stream`, optional) – A stream to be recorded. If not given, the current stream will be used.

Wait for the event to complete. This prevents the CPU thread from proceeding until the event completes.

Make all future work submitted to the given stream wait for this event.

stream (torch.Stream, optional) – A stream to synchronize. If not given, the current stream will be used.